CMPSC 448 Midterm 1 Project

Done Individually

Aman Chatur

October 11, 2026

Problem Definition

The goal of this project is to see if a classifier can tell which LLM family wrote a response. I treated it as a 4 class text classification problem with the families GPT, Claude, Llama, and PaLM. Each example has an LLM name, the user's prompt, and the model's response. I built a CNN and an LSTM, which are the two required models, and used them to answer all four research questions. I also trained a TF-IDF and logistic regression model as an extra baseline to compare against.

Dataset

Where the data came from

I used the public LMSYS Chatbot Arena Conversations dataset from Hugging Face. On Chatbot Arena, a user types a prompt, two anonymous models answer it, and the user votes for the better answer. The dataset has 33,000 of these conversations from April to June 2023. Each one includes both model names, the prompt, both answers, a timestamp, the language, and a moderation flag. The dataset is gated, so I requested access on Hugging Face and downloaded it with the datasets library.

How I built the dataset

Each conversation gives two examples, one for each model, with the same prompt. I kept only English conversations and only used the first prompt and first reply. I mapped each model to a family as shown in Table 1 and dropped models that did not fit one of the four families, like ChatGLM, MPT, and Dolly. I also removed very short answers and duplicates.

The families were very unbalanced, so I randomly sampled each one to the size of the smallest family, PaLM, which had 2,604 answers. I tagged each prompt as coding, math, writing, or general using keyword rules for RQ3. Finally, I split the data 70/10/20 into train, validation, and test sets, making sure the same prompt never shows up in more than one split. Otherwise a model could just memorize prompts it already saw.

Family	Models included	Rows before balancing
GPT	gpt-4, gpt-3.5-turbo	7,635
Claude	claude-v1, claude-instant-v1	5,604
Llama	llama-13b, vicuna-13b, vicuna-7b, koala-13b, alpaca-13b, wizardlm-13b, guanaco-33b, gpt4all-13b-snoozy	20,513
PaLM	palm-2	2,604

Table 1. Model to family mapping.

Family	Train	Val	Test	Total	Coding	Math	Writing	General
GPT	1,839	246	519	2,604	303	138	307	1,856
Claude	1,817	258	529	2,604	321	152	320	1,811
Llama	1,834	261	509	2,604	282	127	299	1,896
PaLM	1,787	265	552	2,604	318	182	347	1,757
Total	7,277	1,030	2,109	10,416	1,224	599	1,273	7,320

Table 2. Final dataset sizes by split and task domain.

CNN and RNN Implementation and Training

Preprocessing

I split the text into words, punctuation marks, and line breaks, and kept capitalization. I kept punctuation and line breaks on purpose because formatting bullet points and bold text is part of each model's style. The vocabulary was built from the training set only and has 28,734 tokens. Responses were cut to 384 tokens, prompts to 128 tokens, and prompt plus response to 512 tokens.

CNN

The CNN starts with a 128-dimensional embedding layer. It then runs three convolution layers side by side with filter widths of 3, 4, and 5 tokens and 128 filters each. Each filter learns to detect a short phrase. Max pooling keeps the strongest match for each filter anywhere in the text, and then dropout of 0.5 and a linear layer give the 4 class scores.

LSTM

The RNN is a 2-layer bidirectional LSTM with a hidden size of 128 in each direction, using the same embedding size. Reading the text in both directions lets it use the whole response, like how it starts, how it is laid out, and how it ends. I combined the outputs from every step using both max pooling and mean pooling, then used dropout and a linear layer like the CNN.

Training

Both models used the same training setup: the Adam optimizer with a learning rate of 0.001, cross entropy loss, batch size 64, and up to 15 epochs. After every epoch I checked macro F1 on the validation set, kept the best version, and stopped early if it did not improve for 3 epochs. I trained everything on my local computer using PyTorch. The CNN was much faster, taking about 7 seconds for RQ1 compared to about 103 seconds for the LSTM.

RQ1: Can we identify which LLM generated a response?

For RQ1 I trained both models on the response text only. Random guessing would get 0.25 since there are four balanced classes.

Model	Accuracy	Macro F1	F1 GPT	F1 Claude	F1 Llama	F1 PaLM
CNN	0.706	0.705	0.556	0.875	0.586	0.804
LSTM	0.697	0.699	0.598	0.878	0.522	0.796
TF-IDF baseline	0.710	0.708	-	-	-	-

Table 3. RQ1 test results using the response only.

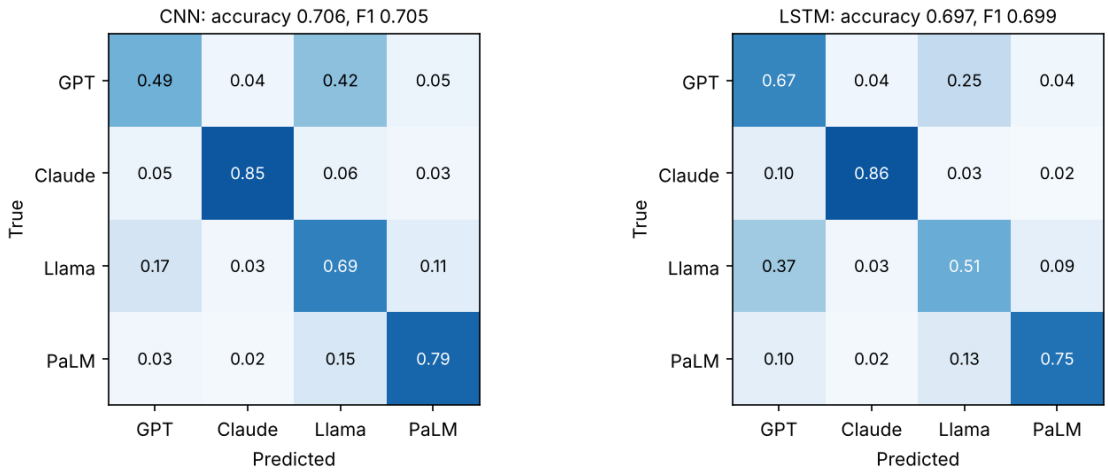


Figure 1. Confusion matrices for the CNN on the left and the LSTM on the right.

Yes, the LLM can be identified from its response. Both models got about 70% accuracy, which is almost three times better than guessing. The CNN and LSTM performed about the same, but the CNN trained about 14 times faster.

Claude was the easiest family to identify, followed by PaLM. GPT and Llama were the hardest, and Figure 1 shows they were mostly confused with each other. The CNN labeled 42% of GPT answers as Llama. This makes sense because several of the Llama models, like Vicuna, Alpaca, and GPT4All, were trained on ChatGPT outputs, so they write a lot like GPT.

RQ2: Does the user's prompt help identify the LLM?

For RQ2 I trained each model three times: on the prompt only, the response only, and both together.

Input	CNN Acc	CNN F1	LSTM Acc	LSTM F1	TF-IDF Acc	TF-IDF F1
Prompt only	0.279	0.271	0.270	0.269	0.269	0.269
Response only	0.706	0.705	0.697	0.699	0.710	0.708
Prompt + response	0.703	0.698	0.676	0.676	0.693	0.690

Table 4. RQ2 test results for each input type.

Can the LLM be predicted from the prompt alone?

Not really. Using only the prompt, every model scored around 0.27, which is barely above guessing. This is expected because on Chatbot Arena the user writes the prompt first and the models are picked at random after.

Does adding the prompt help?

No. Adding the prompt to the response slightly lowered the score for every model. The prompt has almost no information about which model answered, so it just adds noise.

Why is prompt only slightly above chance?

The small edge comes from the dataset, not the prompts themselves. Figure 2 shows that different models were active on Arena at different times. The first week only has Llama models, PaLM does not show up until May 8, and PaLM is almost half the data in the week of May 15. Since the topics people asked about

also changed over time, a prompt can hint at when it was asked, which hints at which models were active. This is a bias in how the data was collected, not a real fingerprint.

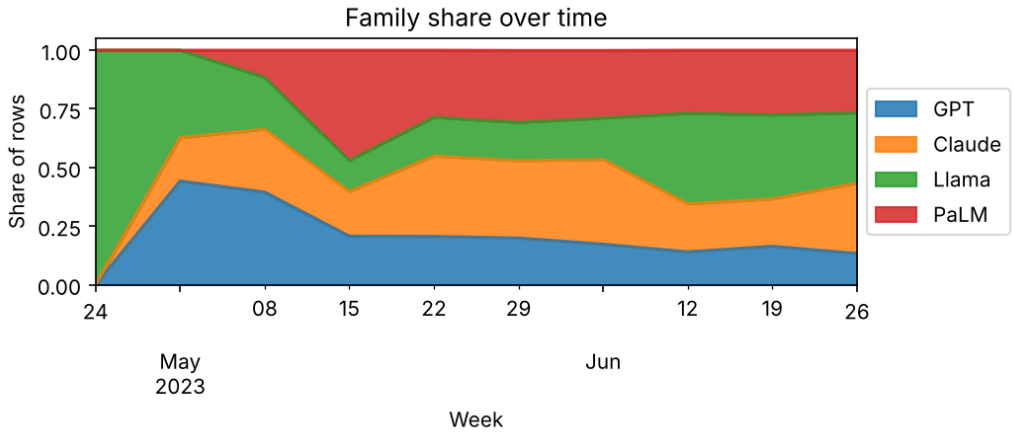


Figure 2. Share of each family in the dataset by week.

RQ3: Do LLM fingerprints generalize across task domains?

For RQ3 I held out one domain at a time, first writing and then coding. In the cross domain setting, the model trains on every other domain and is tested only on the held out one. In the in domain setting, the model trains on a random sample of the same size that does include that domain. Comparing the two shows how much the fingerprint depends on the task.

Test domain	Model	Setting	Acc	Macro F1	F1 GPT	F1 Claude	F1 Llama	F1 PaLM
Writing	CNN	Cross domain	0.618	0.603	0.500	0.748	0.462	0.703
Writing	CNN	In domain	0.723	0.714	0.634	0.917	0.476	0.828
Writing	LSTM	Cross domain	0.660	0.654	0.561	0.821	0.455	0.779
Writing	LSTM	In domain	0.723	0.723	0.639	0.885	0.556	0.812
Coding	CNN	Cross domain	0.644	0.635	0.605	0.738	0.490	0.709
Coding	CNN	In domain	0.716	0.710	0.662	0.816	0.606	0.758
Coding	LSTM	Cross domain	0.701	0.693	0.656	0.825	0.552	0.736
Coding	LSTM	In domain	0.709	0.709	0.649	0.794	0.619	0.772

Table 5. RQ3 results. The writing test set has 238 examples and the coding test set has 261.

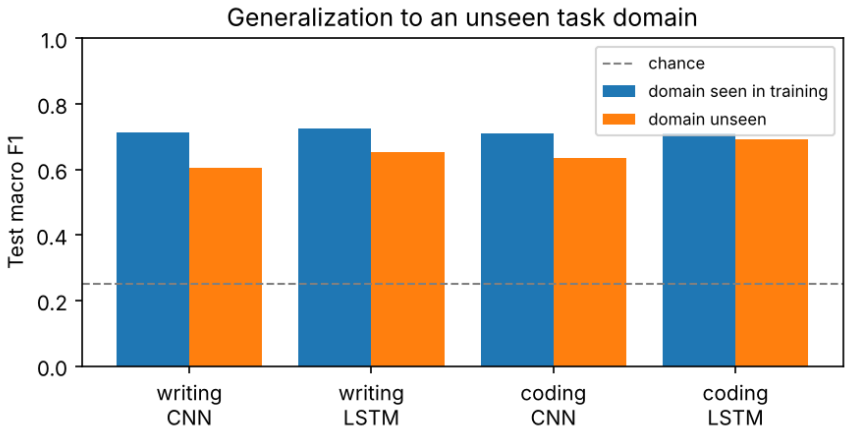


Figure 3. Macro F1 when the test domain was seen or unseen during training.

Does performance drop on unseen tasks?

Yes, in every case the score dropped when the domain was not seen in training. The CNN dropped by 0.11 on writing and 0.08 on coding, and the LSTM dropped by 0.07 and 0.02. Even so, all scores stayed between 0.60 and 0.69, which is still far above guessing.

Are some models easier to identify across domains?

Claude and PaLM were easy to identify in every setting. Llama was the hardest in every setting and barely changed between cross domain and in domain, so its problem is being similar to GPT, not the task.

General fingerprint or task specific patterns?

The models learned mostly a general fingerprint with some task specific patterns on top. The LSTM handled new domains better than the CNN, which suggests the CNN relies more on topic phrases while the LSTM picks up the overall style of a response. The test sets are small though, so small differences could just be noise.

RQ4: What characteristics distinguish different LLMs?

For RQ4 I measured 20 simple style features for each response, trained a random forest on only those features, looked at which words the TF-IDF model linked to each family, and retrained the CNN and LSTM after removing different signals from the text.

Style features by family

Feature	GPT	Claude	Llama	PaLM
Words per response	125.2	162.9	119.6	168.7
Lines per response	10.2	15.3	7.8	12.8
Average word length	4.82	5.21	4.63	4.45
Average sentence length	20.4	20.9	23.7	20.5
Vocabulary diversity	0.718	0.676	0.676	0.640
Bullet point lines	0.44	2.43	0.36	1.96
Numbered list lines	1.68	1.44	1.23	0.61
Bold text markers	0.01	0.01	0.02	0.74
Starts with Sure or Certainly	6.0%	0.7%	6.0%	10.4%
Says As an AI	8.0%	1.2%	4.4%	7.1%
Contains an apology	3.4%	6.9%	4.7%	1.5%

Table 6. Average style features per response for each family.

Each family has its own style. Claude writes long answers with the most lines and bullet points and apologizes the most, but rarely says "As an AI." PaLM also writes long answers and uses bold formatting much more often than the other families. GPT has the most varied vocabulary, uses numbered lists, and says "As an AI" the most. Llama writes the shortest answers with the fewest line breaks.

Which features matter most?

The random forest got 0.606 accuracy using only the 20 style features. That is lower than the neural models but still far above guessing, so a lot of the signal comes from simple features. Figure 4 shows the most important ones were the number of lines, the number of colons, vocabulary diversity, word length, and sentence length. Most of these are about structure.

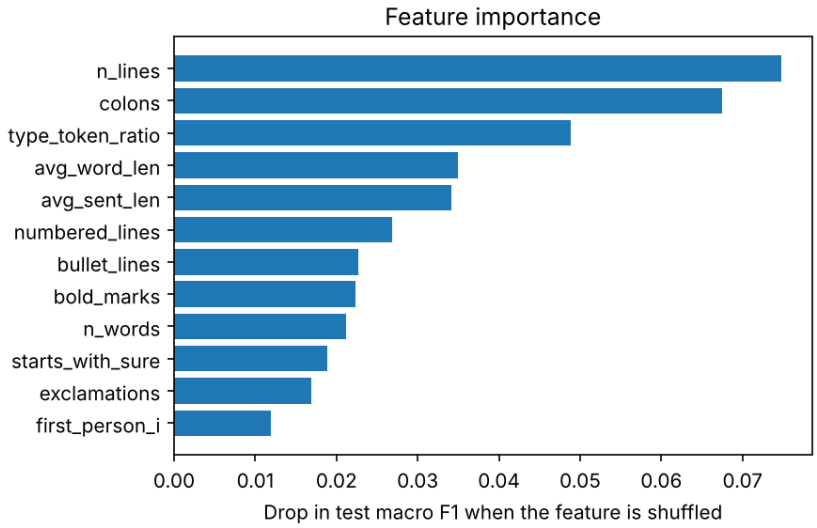


Figure 4. How much macro F1 drops when each feature is shuffled.

Which words point to each family?

Family	Most indicative words and symbols
Claude	• bullet, Here is, So, summary, apologize, line breaks
GPT	- bullet, As an AI, AI language, various, provide, assist
Llama	and the, such as, the, used to, It
PaLM	* bullet and bold, also, is a, Here, The, that is

Table 7. Top words and symbols for each family from the TF-IDF model.

These match the features. Claude uses the • bullet and "Here is," GPT uses dash bullets and "As an AI language model," and PaLM uses * for bullets and bold text. Llama's top words are just common words like "and the" and "such as," meaning it has no strong habit of its own, which is why it is the hardest to identify.

Does removing signals hurt performance?

Text given to the model	CNN F1	Change	LSTM F1	Change
Full text	0.705	-	0.699	-
Formatting removed	0.676	-0.029	0.666	-0.033
First 50 words only	0.593	-0.112	0.626	-0.073
Lowercase words only	0.645	-0.060	0.612	-0.086
First 10 words removed	0.623	-0.083	0.597	-0.102

Table 8. Results after removing each signal and retraining.

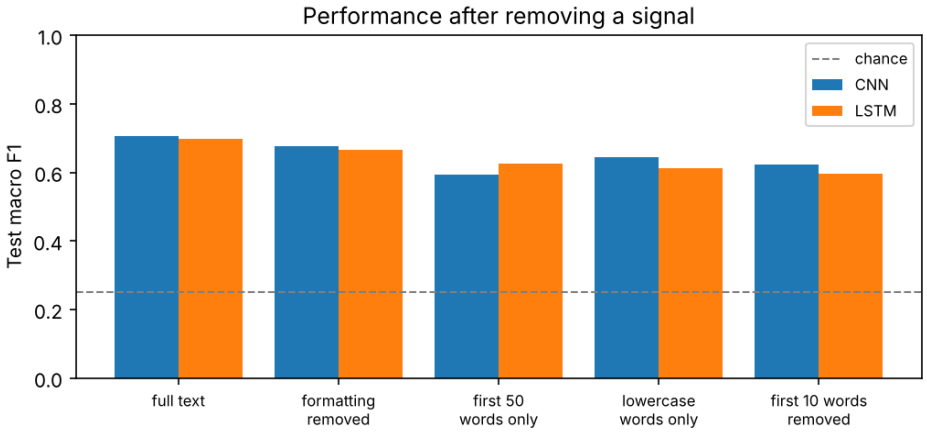


Figure 5. Macro F1 after removing each signal.

Removing any signal lowered the score, but none brought it close to guessing. Cutting responses to 50 words hurt the CNN the most, so length is a big clue. Removing the first 10 words hurt the LSTM the most, which shows the way a response opens, like "Sure!" or "As an AI," is a strong clue. Removing formatting only cost about 0.03 because the models could still use word choice. Overall the models use both structure and word choice, and the fingerprint is spread across many signals instead of depending on just one.

Lessons and Experience

The biggest lesson was that how you split the data matters as much as the model. The same prompt is shown to two models on Arena, so I had to split by prompt to make sure test prompts were never seen in training.

I also learned that simple baselines are worth running. The TF-IDF model scored about the same as both neural networks on RQ1, which showed that most of the signal is in word and symbol choice. Between the two required models, the CNN was the better choice for speed since it matched the LSTM and trained much faster, but the LSTM handled new domains better.

Another lesson was that the way labels are defined changes the problem. Grouping Vicuna and Alpaca with Llama made sense since they are built on LLaMA, but they were trained on ChatGPT outputs, which

made GPT and Llama hard to tell apart. I also saw that dataset artifacts can look like real signal, since the prompt only result in RQ2 came from when models were active, not from the prompts.

Finally, balancing the classes by downsampling threw away about 70% of the data. In the future I would try class weights instead. The models in this dataset are also from 2023, so the results might not hold for newer models.